

Django

TomerBu

נושאים להיום:

Django Rest Framework

מבוא

התקנה והגדרות

Request Response Classes

Serializers

Function Based Views

Class Based Views

Django REST Framework

<https://www.django-rest-framework.org/>

DRF

The image shows a screenshot of the Django REST Framework website's navigation menu. The menu is dark-themed with white text. It features three main categories: 'Tutorial', 'API Guide', and 'Topics'. The 'Tutorial' category is expanded, showing a list of six items: 'Quickstart', '1 - Serialization', '2 - Requests and responses', '3 - Class based views', '4 - Authentication and permissions', and '5 - Relationships and hyperlinked APIs'. The 'API Guide' category is also expanded, showing a list of 18 items: 'Requests', 'Responses', 'Views', 'Generic views', 'Viewsets', 'Routers', 'Parsers', 'Renderers', 'Serializers', 'Serializer fields', 'Serializer relations', 'Validators', 'Authentication', 'Permissions', 'Caching', 'Throttling', 'Filtering', 'Pagination', 'Versioning', 'Content negotiation', 'Metadata', 'Schemas', and 'Format suffixes'.

Tutorial ▼ **API Guide ▼** **Topics ▼**

Quickstart

- 1 - Serialization
- 2 - Requests and responses
- 3 - Class based views
- 4 - Authentication and permissions
- 5 - Relationships and hyperlinked APIs
- 6 - Viewsets and routers

API Guide ▼ **Topics**

- Requests
- Responses
- Views
- Generic views
- Viewsets
- Routers
- Parsers
- Renderers
- Serializers
- Serializer fields
- Serializer relations
- Validators
- Authentication
- Permissions
- Caching
- Throttling
- Filtering
- Pagination
- Versioning
- Content negotiation
- Metadata
- Schemas
- Format suffixes

Django REST Framework

<https://www.django-rest-framework.org/>

DRF

```
# Create the project directory
```

```
mkdir lec4
```

```
cd lec4
```

יצירת תיקיה חדשה ופתיחה שלה בVSCode

```
code .
```

```
# Create a virtual environment to isolate our package dependencies locally
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate # On Windows use `.venv\Scripts\activate`
```

```
# Install Django and Django REST framework into the virtual environment
```

```
pip install djangorestframework
```

```
# Set up a new project with a single application
```

```
django-admin startproject lec4 . # Note the trailing '.' character
```

```
django-admin startapp api
```

Django REST Framework

התקנה:

```
python manage.py startapp api
```

```
pip install djangorestframework
```

Add `'rest_framework'` to your `INSTALLED_APPS` setting.

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
]
```

Django REST Framework

יצירת משתמש מנהל:

```
python manage.py migrate
```

```
python manage.py createsuperuser
```

סיימנו עם ההתקנות: צריך לחבר את האפליקציה api:

lec4/api/views.py

```
from django.http import HttpResponse
from django.shortcuts import render

# Create your views here.
def index(request):
    return HttpResponse("it works")
```

lec4/api/urls.py

```
from api.views import index
from django.urls import path

urlpatterns = [
    path('', index, name='index')
]
```

lec4/lec4/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls'))
]
```

Django REST Framework

התקנה:

api/views.py

```
from django.shortcuts import render
from django.http import HttpResponse

def index(request):
    return HttpResponse("ok")
```

python manage.py runserver

lec3/api/urls.py

```
from django.urls import path
from .views import index

urlpatterns = [
    path('students/', index, name='students'),
]
```

ניצור קובץ urls

בדיקה:

http:localhost:8000/api/students

נערוך את קובץ urls של lec3

lec3/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

מבוא - למה צריך DRF?

api/views.py

```
from django.http import HttpResponseRedirect, JsonResponse
from django.shortcuts import render
```

```
class Student():
    def __init__(self, name, age):
        self.name = name
        self.age = age
s1 = Student("Dani", 20)
```

```
# Create your views here.
def index(request):
    #return HttpResponseRedirect(s1)
    return JsonResponse(s1)
```

עם Django בלבד
מסורבל להחזיר JSON

Serializer/Deserializer:

serialization:

מחלקה שיודעת להמיר אובייקט ל JSON

deserialization:

מחלקה שיודעת לתרגם JSON לאובייקט של פייתון

Serializer/Deserializer:

lec4/api/serializers.py

```
from rest_framework import serializers
```

```
class Student():  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
s1 = Student("Dani", 20)
```

```
class StudentSerializer(serializers.Serializer):  
    name = serializers.CharField(max_length=100)  
    age = serializers.IntegerField()
```

```
# ctrl+shift+p => python select interpreter
```

יש לנו סטודנט- > רוצים JSON

הפתרון של DRF

נכתוב מחלקה
שתשמש לתרגום

הופכת אובייקט לJSON
לתרגום JSON לאובייקט

ניצור מודל שאותו נחזיר בתגובה:

lec4/api/models.py

```
from django.db import models

# Create your models here.
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return f'{self.name} - {self.age}'
```

lec4/api/admin.py

```
from django.contrib import admin

from api.models import Student

# Register your models here.

admin.site.register([Student])
```

python manage.py makemigrations api

python manage.py migrate

python manage.py runserver

ניצור מודל שאותו נחזיר בתגובה:

lec4/api/serializers.py

```
from rest_framework import serializers

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
    age = serializers.IntegerField()
```

lec4/api/views.py

Django REST Framework

מבוא לצורך:

api/views.py

```
from django.shortcuts import render
from django.http import HttpResponse, JsonResponse

# return JsonResponse
def index(request):
    return JsonResponse({'message': 'Hello, World!'})
```

← → ↺ 🏠 ⓘ localhost:8000/api/students/

🗄️ | Hackeru - Hackam... | Priza-Net (v. 4.5) - 🌈 Designing INSTEA

```
1 // 20250218112646
2 // http://localhost:8000/api/students/
3
4 {
5     "message": "Hello, World!"
6 }
```

Django REST Framework

מבוא לצורך:

```
from django.http import JsonResponse
from .models import Student

# return JsonResponse
def index(request):
    students = Student.objects.all()
    return JsonResponse({'message': students})
```

TypeError at /api/students/

Object of type QuerySet is not JSON serializable

Request Method: GET

Request URL: http://localhost:8000/api/students/

Django Version: 5.1.6

Exception Type: TypeError

Exception Value: Object of type QuerySet is not JSON serializable

Exception Location: /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/json/encoder.py, line 180, in default

Raised during: api.views.index

Python Executable: /Volumes/E4TB/W150824MR/Django/lec3/.venv/bin/python

Python Version: 3.13.1

Python Path: ['/Volumes/E4TB/W150824MR/Django/lec3',
'/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13.zip',
'/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13',
'/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/lib-dynload',
'/Volumes/E4TB/W150824MR/Django/lec3/.venv/lib/python3.13/site-packages']

Server time: Tue, 18 Feb 2025 09:37:39 +0000

פורמט להעברת נתונים:

פורמט להעברת נתונים בין השפות השונות:
בבקשת HTTP נשלח JSON או XML

```
1 <song position="6">↵  
2   <artist>65daysofstatic</artist>↵  
3   <title>Come to Me</title>↵  
4   <album>We Were Exploding Anyway</album>↵  
5   <release>26 April 2010</release>↵  
6   <duration>8:00</duration>↵  
7   <special>Robert Smith, The Cure</special>↵  
8 </song>
```

סריליזציה:

תרגום של אובייקט משפת תכנות לפורמט רצוי (דוגמא JSON/XML)

דה-סריליזציה:

פענוח של XML/JSON ויצירת אובייקט בשפת התכנות

Serialization and De-Serialization:

מחלקה של Student:
אובייקט מהמחלקה Student

הוא אובייקט בפייתון (הוא לא Json)

Serialization:

המרת אובייקט משפת תכנות לJSON או XML

De-Serialization:

המרת JSON לאובייקט בשפת תכנות

Serialization and De-Serialization:

serializers.py

נגדיר Serializer שיודע לתרגם אובייקט Student ל-JSON:

```
from rest_framework import serializers

# serializers convert objects to json and vice versa

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
```

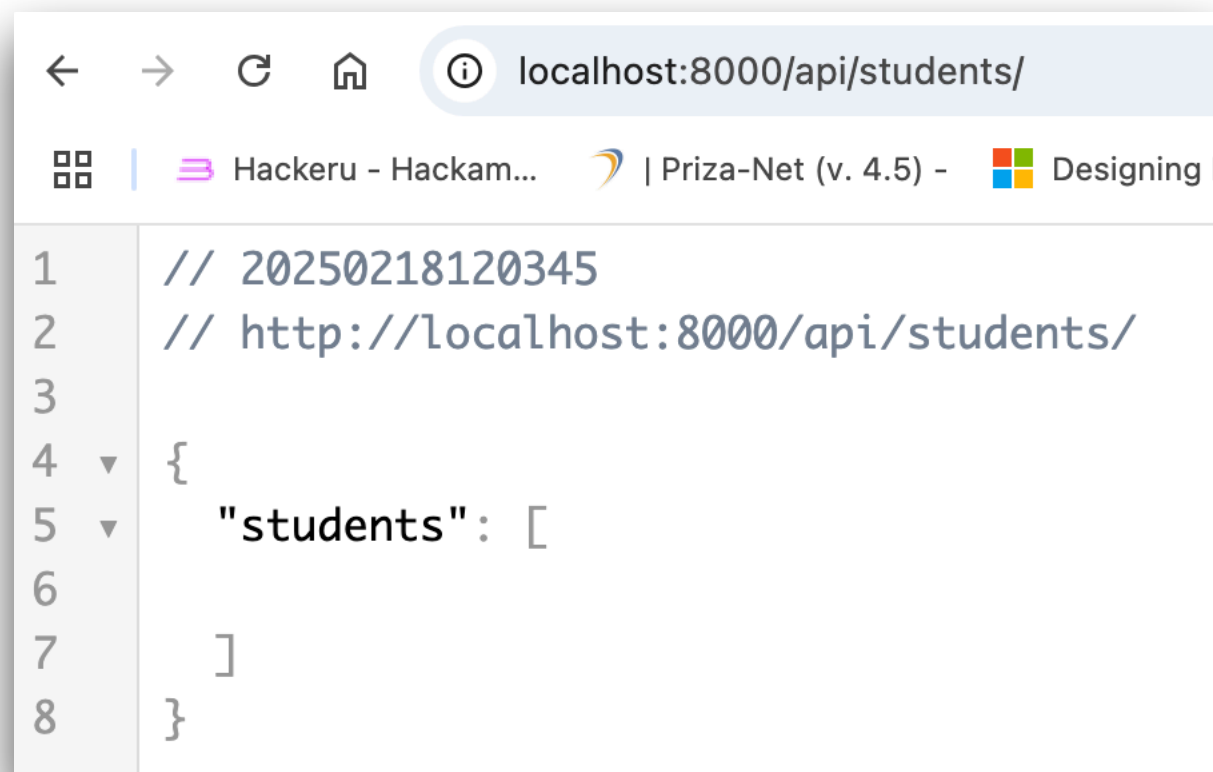
```
from django.http import JsonResponse
from .models import Student
from .serializers import StudentSerializer
```

```
# return JsonResponse
def index(request):
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return JsonResponse({'students': serializer.data})
```

יצירת מופע של StudentSerializer

JSON

Serialization and De-Serialization:



The screenshot shows a web browser window with a REST client interface. The address bar displays 'localhost:8000/api/students/'. The browser tabs include 'Hackeru - Hackam...', 'Priza-Net (v. 4.5)', and 'Designing I'. The main content area shows a JSON response with line numbers 1 through 8 on the left. The JSON is a root object with a 'students' array.

```
1 // 20250218120345
2 // http://localhost:8000/api/students/
3
4 {
5   "students": [
6
7   ]
8 }
```

Serialization and De-Serialization:

POST Serializer יכול לקבל מידע מבקשה של
create() לטובת יצירת מופע חדש - לשם כך קיימת המתודה

PUT Serializer יכול לקבל מידע מבקשה של
update() לטובת יצירת עדכון מופע קיים - לשם כך קיימת המתודה

```
from rest_framework import serializers

# serializers convert objects to json and vice versa
from .models import Student
class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)

    def update(self, instance, validated_data):
        # copy all the props from the request to db object
        instance.name = validated_data['name']

        # save the db object
        instance.save()
        return instance

    def create(self, validated_data):
        # Student.objects.create(name=validated_data['name'])
        return Student.objects.create(**validated_data)
```

Serialization and De-Serialization:

Serializer יכול לקבל מידע מבקשה של POST
לטובת יצירת מופע חדש - לשם כך קיימת המתודה create()

Serializer יכול לקבל מידע מבקשה של PUT
לטובת יצירת עדכון מופע קיים - לשם כך קיימת המתודה update()

```
from rest_framework import serializers

# serializers convert objects to json and vice versa
from .models import Student
class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)

    def update(self, instance, validated_data):
        # copy all the props from the request to db object
        instance.name = validated_data.get('name', instance.name)

        # save the db object
        instance.save()
        return instance

    def create(self, validated_data):
        # Student.objects.create(name=validated_data['name'])
        return Student.objects.create(**validated_data)
```

Request/Response Objects

DRF סורקת את ה View שלנו
ומייצרת עבורנו דוקומנטציה:

כדי שזה יעבוד:
יש לוודא שמחזירים Response
ולא הטיפוסים של Django

כותבים מעל הפונקציה
לאילו http methods

הספרייה יוצרת עבורנו
דוקומנטציה

```
from rest_framework.response import Response
from rest_framework.request import Request
from rest_framework.decorators import api_view
from .models import Student
from .serializers import StudentSerializer

@api_view(['GET'])
def index(request: Request):
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response({'students': serializer.data})
```

Browsable API:

Django REST framework

Index

Index

OPTIONS GET

GET /api/students/

HTTP 200 OK
Allow: OPTIONS, GET, POST
Content-Type: application/json
Vary: Accept

{
 "students": []
}

Media type:
application/json

Content:
{
 |
}

POST

GET/POST/PUT/DELETE

```
from rest_framework import serializers
from api.models import Student
class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
    age = serializers.IntegerField()

    # POST
    def create(self, validated_data):
        return Student.objects.create(**validated_data)

    # PUT
    def update(self, instance, validated_data):
        instance.name = validated_data.get('name', instance.name)
        instance.age = validated_data.get('age', instance.age)
        instance.save()
        return instance
```

בהמשך נלמד מחלקה של Serializer שעושה הכל במקומנו.

אנחנו לומדים כרגע את המהות של Serializer:

- (1) מתרגם JSON
- (2) מתודות יצירה ועדכון כנגד הדטאבייס
- (3) ולידציה

התמודדות עם בקשת POST

דה-סריליזציה:

מקבל JSON מהלקוח וממיר לאובייקט

```
@api_view(['GET', 'POST'])
def index(request: Request):

    if request.method == 'POST':
        # serializer is in create mode:
        serializer = StudentSerializer(data = request.data)
        if serializer.is_valid():
            serializer.save() # will invoke create
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)

    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response({'students': serializer.data})
```

סטטוס לבקשת POST מוצלחת

התמודדות עם בקשת POST

Index

OPTIONS

GET

GET /api/students/

HTTP 200 OK

Allow: POST, OPTIONS, GET

Content-Type: application/json

Vary: Accept

```
{
  "students": [
    {
      "name": "Larry"
    },
    {
      "name": "Larry"
    },
    {
      "name": "L"
    }
  ]
}
```

Media type:

application/json

Content:

{"name": "Larry"}

Make a POST request on
Index resource

POST

בדיקת תקינות - validation

```
from rest_framework import serializers
from api.models import Student
from django.core.validators import MinValueValidator, MaxValueValidator

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
    age = serializers.IntegerField(
        validators=[MinValueValidator(4), MaxValueValidator(120)]
    )

    def create(self, validated_data):
        return Student.objects.create(**validated_data)

    def update(self, instance, validated_data):
        instance.name = validated_data.get('name', instance.name)
        instance.age = validated_data.get('age', instance.age)
        instance.save()
        return instance
```

(1) מתרגם JSON

(2) מתודות יצירה ועדכון כנגד הדטהבייס

(3) ולידציה

פעולות CRUD

```
from rest_framework import status
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework.decorators import api_view

from api.models import Student
from api.serializers import StudentSerializer

@api_view(['GET', 'POST'])
def index(request: Request):
    if request.method == 'POST':
        # request.data (body)
        serializer = StudentSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save() # will invoke create
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)

    # GET
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response({'students': serializer.data})

@api_view(['GET', 'PUT', 'DELETE'])
def details(request: Request, id: int):

    if request.method == "GET":
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student)
        return Response(serializer.data)

    if request.method == "DELETE":
        student = Student.objects.get(id=id)
        student.delete()
        return Response({'message': 'deleted'}, status=status.HTTP_200_OK)

    if request.method == "PUT":
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student, request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_200_OK)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    return Response(status=status.HTTP_405_METHOD_NOT_ALLOWED)
```

פעולות CRUD

```
from rest_framework import status
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework.decorators import api_view

from api.models import Student
from api.serializers import StudentSerializer

@api_view(['GET', 'POST'])
def index(request: Request):
    if request.method == 'POST':
        # request.data (body)
        serializer = StudentSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save() # will invoke create
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)

    # GET
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response({'students': serializer.data})

@api_view(['GET', 'PUT', 'DELETE'])
def details(request: Request, id: int):

    if request.method == "GET":
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student)
        return Response(serializer.data)

    if request.method == "DELETE":
        student = Student.objects.get(id=id)
        student.delete()
        return Response({'message': 'deleted'}, status=status.HTTP_200_OK)

    if request.method == "PUT":
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student, request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_200_OK)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    return Response(status=status.HTTP_405_METHOD_NOT_ALLOWED)
```

תרגול פעולות :CRUD

- 1 צרו מודל חדש בשם Kitten
בקובץ models.py
תכונות: name, breed מחרוזות בגודל מקסימלי של 100 תווים
- 2 רשמו את המודל
בקובץ admin.py
- 3 python manage.py makemigrations api
python manage.py migrate
- 4 כתבו Serializer עבור חתולים
- 5 כתבו 2 פונקציות בקובץ views.py
אחת ל-GET אחת ל-POST לפי ID בדומה למה שעשינו עם students
- 6 הוסיפו 2 נתיבים בקובץ urls.py
/kittens
/kittens/id
- 7 יש להוסיף בדיקות עם Postman

Model Serializer

ModelSerializer

Often you'll want serializer classes that map closely to Django model definitions.

The `ModelSerializer` class provides a shortcut that lets you automatically create a `Serializer` class with fields that correspond to the Model fields.

The `ModelSerializer` class is the same as a regular `Serializer` class, except that:

- It will automatically generate a set of fields for you, based on the model.
- It will automatically generate validators for the serializer, such as `unique_together` validators.
- It includes simple default implementations of `.create()` and `.update()`.

Declaring a `ModelSerializer` looks like this:

```
class AccountSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Account  
        fields = ['id', 'account_name', 'users', 'created']
```

<https://www.django-rest-framework.org/api-guide/serializers/#modelserializer>

Model Serializer

```
from rest_framework import serializers
from api.models import Kitten, Student
from django.core.validators import MinValueValidator, MaxValueValidator

class KittenSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kitten
        fields = ['id', 'name', 'breed']
        # fields = '__all__'
        # exclude = ['id']
```

<https://www.django-rest-framework.org/api-guide/serializers/#modelserializer>

Model Serializer

```
from rest_framework import serializers
from api.models import Kitten, Student
from django.core.validators import MinValueValidator, MaxValueValidator

class KittenSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kitten
        fields = ['id', 'name', 'breed']
        # fields = '__all__'
        # exclude = ['id']
```

<https://www.django-rest-framework.org/api-guide/serializers/#modelserializer>

בדיקת תקינות של נתונים בבקשה:

```
from rest_framework import serializers

# serializers convert objects to json and vice versa
from .models import Student
from django.core.validators import MinLengthValidator

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100, validators = [
        MinLengthValidator(2)
    ])
    id = serializers.IntegerField(read_only=True)

    def update(self, instance, validated_data):
        instance.name = validated_data.get('name', instance.name)
        instance.save()
        return instance

    def create(self, validated_data):
        return Student.objects.create(**validated_data)
```

עוד ולידציות:

```
from rest_framework import serializers

# serializers convert objects to json and vice versa
from .models import Student
from django.core.validators import MinLengthValidator, RegexValidator

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField(
        max_length=100,
        validators=[
            MinLengthValidator(2),
            RegexValidator(
                regex='^[a-zA-Z0-9]+$',
                message='name must be alphanumeric',
                code='invalid_name'
            )
        ],
        error_messages = {
            'required': 'Please provide a name for the student',
        }
    )

    id = serializers.IntegerField(read_only=True)

    def update(self, instance, validated_data):
        instance.name = validated_data.get('name', instance.name)
        instance.save()
        return instance

    def create(self, validated_data):
        return Student.objects.create(**validated_data)
```

בקשות PUT/DELETE/GET By id

```
from django.urls import path
from .views import index, edit_student
```

```
urlpatterns = [
    path('students/', index, name='students'),
    path('students/<int:id>', edit_student, name='students'),
]
```

```
@api_view(['GET', 'PUT', 'DELETE']) #/students/3
def edit_student(request: Request, id:int):
    if request.method == 'GET':
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student)
        return Response(serializer.data)

    if request.method == 'DELETE':
        student = Student.objects.get(id=id)
        student.delete()
        return Response({'message' : 'Student Deleted'}, status=204)

    if request.method == 'PUT':
        student = Student.objects.get(id=id)
        serializer = StudentSerializer(student, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=400)
```

מודל חדש לדוגמאות הבאות:

lec3/api/models.py

```
from django.db import models

class Kitten(models.Model):
    name = models.CharField(max_length=100)
    breed = models.CharField(max_length=100)
```

כדי לעדכן את מסד הנתונים - נבנה מיגרציה וניישם אותה:

```
python manage.py makemigrations api
```

```
python manage.py migrate
```

ModelSerializer:

במקום שניצור Serializer במלואו כפי שלמדנו -
DRF יכולה ליצור אותו עבורנו :)

הספרייה תממש עבורנו Serializer לפי ההגדרות במודל:

lec3/api/serializers.py

```
from rest_framework import serializers
from api.models import Kitten

class KittenSerializer(serializers.ModelSerializer):

    class Meta:
        model = Kitten
        # fields = '__all__'
        fields = ['id', 'name', 'breed']
        # exclude = ['id']
```

Class Based Views:

```
from rest_framework.views import APIView
class KittensView(APIView):
    def get(self, request):
        return Response("GET")

    def post(self, request):
        return Response("POST")
```

```
from api.views import index, details, KittensView
from django.urls import path

urlpatterns = [
    path('students/', index, name='index'),
    path('kittens/', KittensView.as_view(), name='kittens'),
    path('students/<int:id>/', details, name='student_detail'),
]
```

עוד קומפוננטה של DjangoRestFramework Class Based Views:

lec3/api/views.py

```
from rest_framework.views import APIView

class KittenViews(APIView):
    def get(self, request):
        # handle get request
        return Response({"message": "kittens"})

    def post(self, request):
        # handle the request
        return Response({"message": "kittens"})
```

ארגון של הקוד באופן מונחה עצמים:

מחלקה עם מתודות
במקום פונקציה אחת משפטי תנאי:

מימוש של מחלקה שיורשת מAPIView
המתודות שניתן לדרוס:

()get
()put
()delete
()post

lec3/api/urls.py

```
from django.urls import path
from .views import index, edit_student, KittenViews

urlpatterns = [
    path('students/', index, name='students'),
    path('students/<int:id>', edit_student, name='student actions'),
    path('kittens/', KittenViews.as_view(), name='kittens')
]
```

APIView:

סידור מונחה עצמים של הקוד שכתבנו קודם ב:Function Based Views

```
class KittenViews(APIView):  
  
    def get(self, request):  
        kittens = Kitten.objects.all()  
        serializer = KittenSerializer(kittens, many = True)  
        return Response({"kittens": serializer.data})  
  
    def post(self, request):  
        kitten = KittenSerializer(data = request.data)  
  
        if kitten.is_valid():  
            kitten.save() # will invoke create of the serializer  
            return Response(kitten.data, status = 201)  
        else:  
            return Response({"errors": kitten.errors}, status = 400)
```

בשיעורים הבאים נראה

שיש מחלקות שיורשות מAPIView ומוסיפות מימוש אוטומטי למתודות

השוואה בין APIView לבין Function Based View

```
@api_view(['GET', 'POST'])
def index(request: Request):

    if request.method == 'POST':
        serializer = StudentSerializer(data = request.data)
        if serializer.is_valid():
            serializer.save() # will invoke create
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)

    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response({'students': serializer.data})
```

מה יותר מסודר בעיניכם (:

OOP VS Functional Programming :-)

```
class KittenViews(APIView):

    def get(self, request):
        kittens = Kitten.objects.all()
        serializer = KittenSerializer(kittens, many = True)
        return Response({"kittens": serializer.data})

    def post(self, request):
        kitten = KittenSerializer(data = request.data)

        if kitten.is_valid():
            kitten.save()
            return Response(kitten.data, status = 201)
        else:
            return Response(kitten.errors, status = 400)
```

שיעורי בית:

צרו API שמחזיר מידע על Fun Facts:

מודל:

FunFact:

תכונות:

text, source, source_url, language, permalink

יש לממש את כל פעולות ה-CRUD
כפי שמימשנו בכיתה

דוגמא ל-API כזה:

<https://uselessfacts.jsph.pl/api/v2/facts/random>

The screenshot shows a Chrome browser window with the address bar displaying `uselessfacts.jsph.pl/api/v2/facts/random`. The page content shows a JSON response from the API. The response includes an ID, a text fact, a source, a source URL, a language, and a permalink.

```
1 // 20250218162407
2 // https://uselessfacts.jsph.pl/api/v2/facts/random
3
4 {
5   "id": "57cb85dcf3078e5b67b63ddc3e733929",
6   "text": "The word \"set\" has more definitions than any other word in the English language.",
7   "source": "djtech.net",
8   "source_url": "http://www.djtech.net/humor/useless_facts.htm",
9   "language": "en",
10  "permalink": "https://uselessfacts.jsph.pl/api/v2/facts/57cb85dcf3078e5b67b63ddc3e733929"
11 }
```